UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Mini Project Report

On

Flight Route Planner

(Multi-Algorithm Flight Path Optimization System)

Submitted by:

Name: Nancy

Uid: 25MCA20253

Class: MCA (General)

Section: 25MCA-2-A

Submitted to:

Ms. Kawaljit Kaur

Associate Professor

Table of Content

S.No.	Content	Page No.
1	Acknowledgement	1
2	Introduction	2
3	Objective	3
4	Technologies Used	4
5	Requirements	5
6	Source Code & Output	6-11
7	Conclusion/Future Work	12
8	References/GitHub Link	13

Acknowledgement

I take this opportunity to express my sincere gratitude to everyone who has supported me throughout this project titled "Flight Route Planner: Multi-Algorithm Flight Path Optimization System".

I am deeply thankful to Ms. Kawaljit Kaur for her valuable guidance, encouragement, and constant support during the development of this project. Her insightful suggestions, motivation, and expert advice have been instrumental in enhancing the quality and success of my work.

I would also like to extend my heartfelt thanks to Chandigarh University and the Department of Master of Computer Applications (MCA) for providing an excellent academic environment, essential resources, and continuous support that made this project possible.

My special thanks go to my classmates, friends, and family members for their unwavering encouragement, patience, and moral support throughout every stage of this project.

Lastly, I wish to acknowledge the valuable assistance of various online learning platforms, GitHub repositories, AI tools such as Copilot, ChatGPT, Gemini, and the official documentation of Python, Matplotlib, Tkinter, and NumPy, which greatly contributed to my learning and successful implementation of this project.

Introduction

The Flight Route Planner is a comprehensive, interactive desktop application developed to visualize and compare multiple graph-based pathfinding algorithms in the context of aviation route optimization. Built using Python with Tkinter for the graphical user interface and Matplotlib for map visualization, the application runs entirely on the client side without requiring any server infrastructure.

This project integrates seven core computer science algorithms:

S No.	Algorithm	Type
1	BFS (Breadth-First Search)	Unweighted Shortest Path
2	DFS (Depth-First Search)	Unweighted Path Finding
3	Dijkstra's Algorithm	Weighted Shortest Path
4	A* Search Algorithm	Heuristic-guided Pathfinding
5	Floyd-Warshall Algorithm	All-Pairs Shortest Paths
6	Prim's Algorithm	Minimum Spanning Tree
7	Kruskal's Algorithm	Minimum Spanning Tree

The application features a real-world airport database with 40+ major international airports, 409 flight routes (edges) connecting airports within 9,000 km, weighted edges representing great-circle distances, and an interactive world map with continent outlines, grid lines, and airport markers.

Users can select origin and destination airports from dropdown menus or by clicking directly on the map, choose any algorithm, and visualize the calculated route with edge weight labels. The system includes a comprehensive results panel showing execution times for each algorithm, route details with hop-by-hop distance accumulation, Minimum Spanning Tree visualization, and a performance benchmark chart comparing all algorithms simultaneously.

Objectives

To develop an interactive flight route visualization system that implements, compares, and visualizes seven different graph-based pathfinding and Minimum Spanning Tree algorithms using real-world airport data and weighted distance metrics.

Specific Objectives

1. **Implement Pathfinding Algorithms** - Implement BFS, DFS, Dijkstra, A*, and Floyd-Warshall algorithms to find optimal routes between airports, each demonstrating different approaches to pathfinding (unweighted, weighted, heuristic-guided, and all-pairs)

2. **Implement Minimum Spanning Tree Algorithms** - Implement Prim's and Kruskal's algorithms to find the minimum network connecting all airports, demonstrating greedy algorithm approaches for network optimization

3. **Build Interactive Map Visualization** - Create an interactive world map with 40+ airports, continent outlines, grid lines, and flight paths using Matplotlib with curved Bézier curves for realistic representation

4. **Implement Real-time Performance Comparison** - Provide live execution time measurements for each algorithm with a side-by-side comparison panel showing milliseconds taken, hops count, and total distance

5. **Display Detailed Route Analytics** - Show comprehensive route details including hop-by-hop distances with cumulative totals, total distance, number of connections, and algorithm-specific metadata

6. **Enable Comparative Algorithm Analysis** - Provide a "Compare All" feature that runs all algorithms simultaneously and displays each route with a unique color on the same map with a legend for easy identification

Technologies Used

The Flight Route Planner is developed using core Python technologies and open-source scientific computing libraries. All dependencies are installed via pip, making the application fully portable across platforms.

S.No.	Technology	Purpose
1	Python 3.8+	Core application logic, algorithm implementation
2	Tkinter	GUI framework, window management, UI components
3	Matplotlib 3.5+	World map rendering, airport visualization, path drawing
4	NumPy	Mathematical computations
5	Heapq	Priority queue implementation for Dijkstra and A*
6	Collections	Deque implementation for BFS algorithm
7	Time	Performance measurement and execution time tracking
8	Datetime	Timestamp generation for status display

Installation Instructions

```bash
# Install required libraries
pip install matplotlib numpy

# Run the application
python flight_route_planner.py
```

Requirements

Hardware Requirements

Component	Minimum
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	200 MB (application + cache)
Display	1366×768 or higher

Software Requirements

Component	Specification
Operating System	Windows 10/11, macOS 11+, Linux (Ubuntu 20.04+)
Python Version	Python 3.8 or higher
Required Libraries	matplotlib, numpy
IDE	VS Code / PyCharm / Any text editor

Source Code & Output

Key Code Snippets

1. BFS Algorithm (Unweighted Shortest Path)

python

```
from collections import deque
def bfs(graph, start, end):
    """Breadth-First Search - finds path with fewest hops"""
queue = deque([[start]])
    visited = {start}
  while queue:
        path = queue.popleft()
        node = path[-1]
      if node == end:
            return path, calculate_path_cost(graph, path)
       for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return [], 0
def calculate_path_cost(graph, path):
    """Calculate total distance of a path"""
    total = 0
    for i in range(len(path) - 1):
        total += graph[path[i]].get(path[i+1], 0)
    return total
```

2. DFS Algorithm (Depth-First Search)

python

```
def dfs(graph, start, end):
    """Depth-First Search - explores deep paths first"""
stack = [[start]]
    visited = set()
    while stack:
        path = stack.pop()
        node = path[-1]
        if node in visited:
            continue
        visited.add(node)
```

```
        if node == end:
            return path, calculate_path_cost(graph, path)
    for neighbor in graph[node]:
        if neighbor not in visited:
            stack.append(path + [neighbor])

    return [], 0
```

Output:-

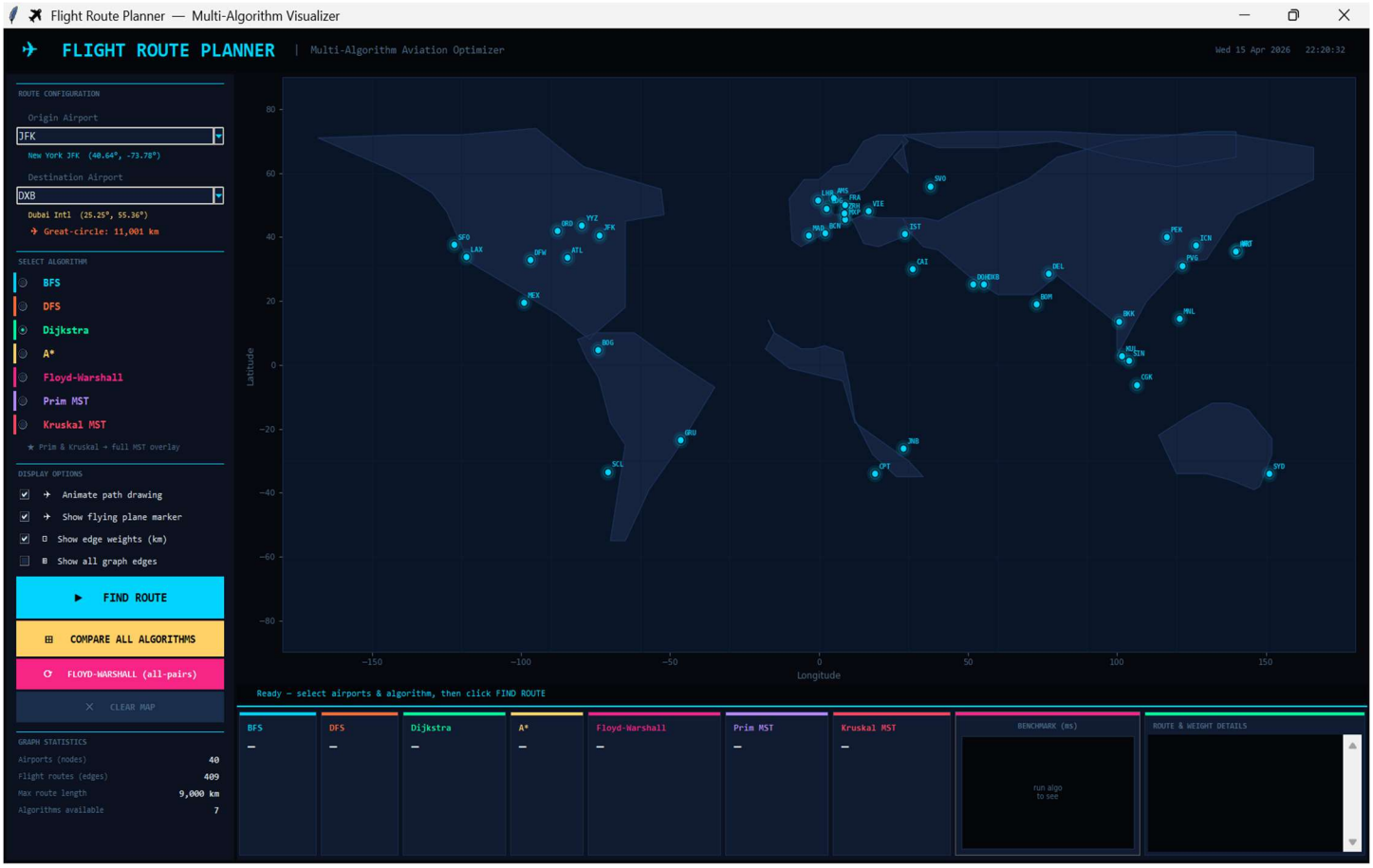
User Interface Layout

The application features a three-panel layout:

Left Sidebar: Route Configuration (Origin/Destination selection), Algorithm Selection (7 algorithms with color coding), Display Options, Action Buttons, Graph Statistics (40 airports, 409 routes)

Center Canvas: Interactive world map with continent outlines, 40+ airports as cyan nodes, latitude/longitude grid, flight routes as curved Bézier paths with edge weights

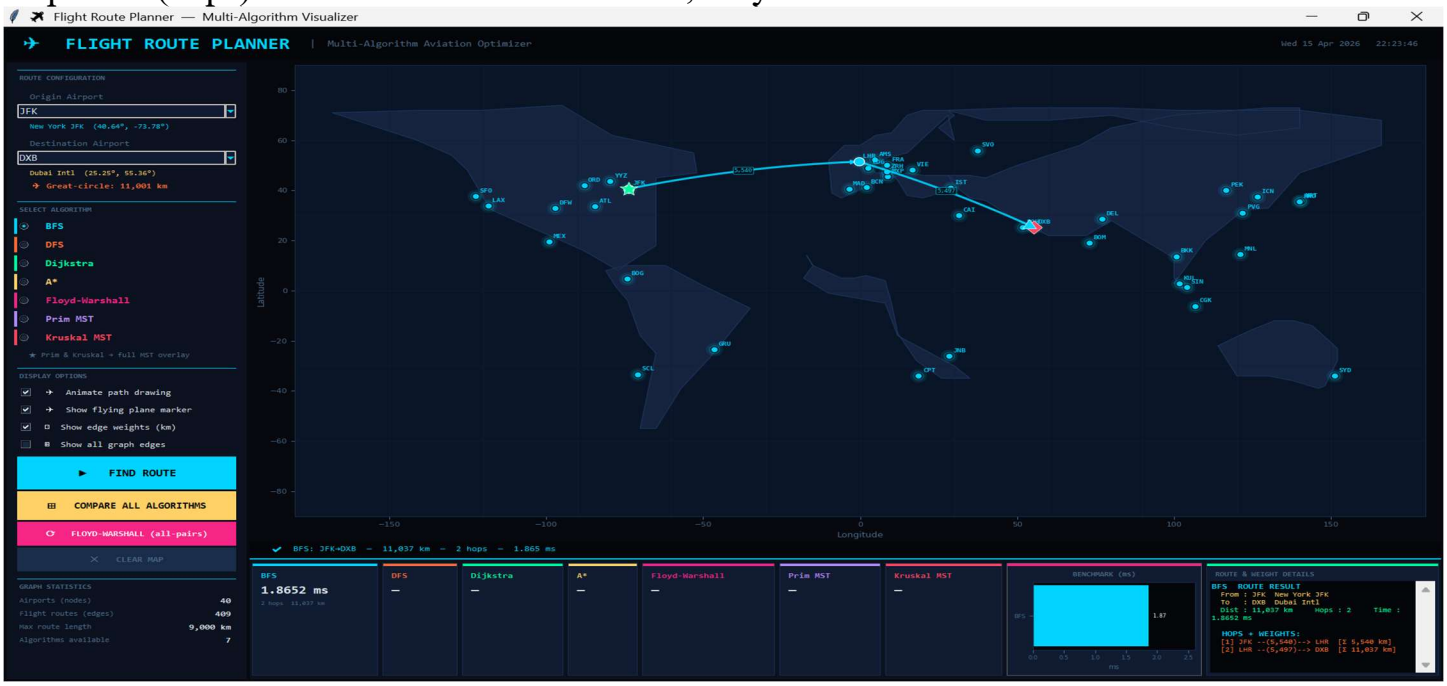
Bottom Panel: Algorithm performance cards showing execution times, benchmark chart, route details text area



Algorithm Output

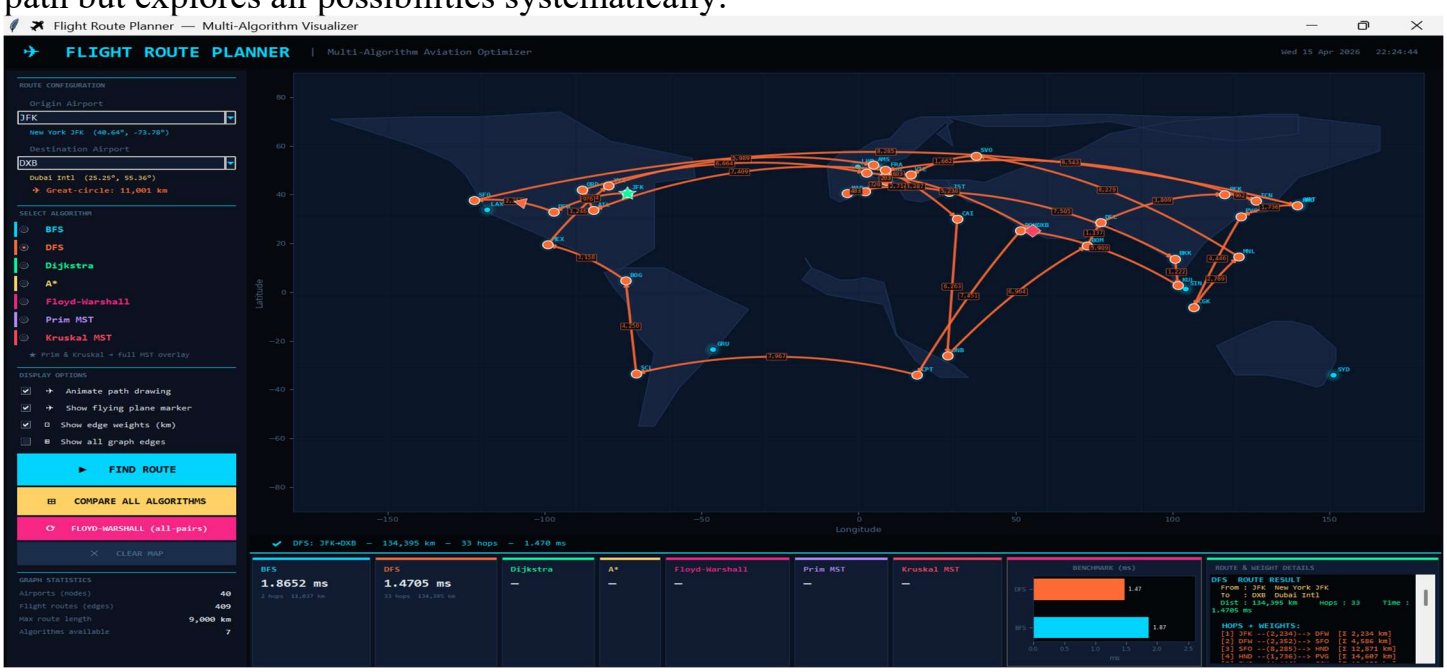
1. BFS (Breadth-First Search)

Explores airports level by level using a queue. Guarantees the path with the fewest number of stopovers (hops). Does not consider distances, only connection count.



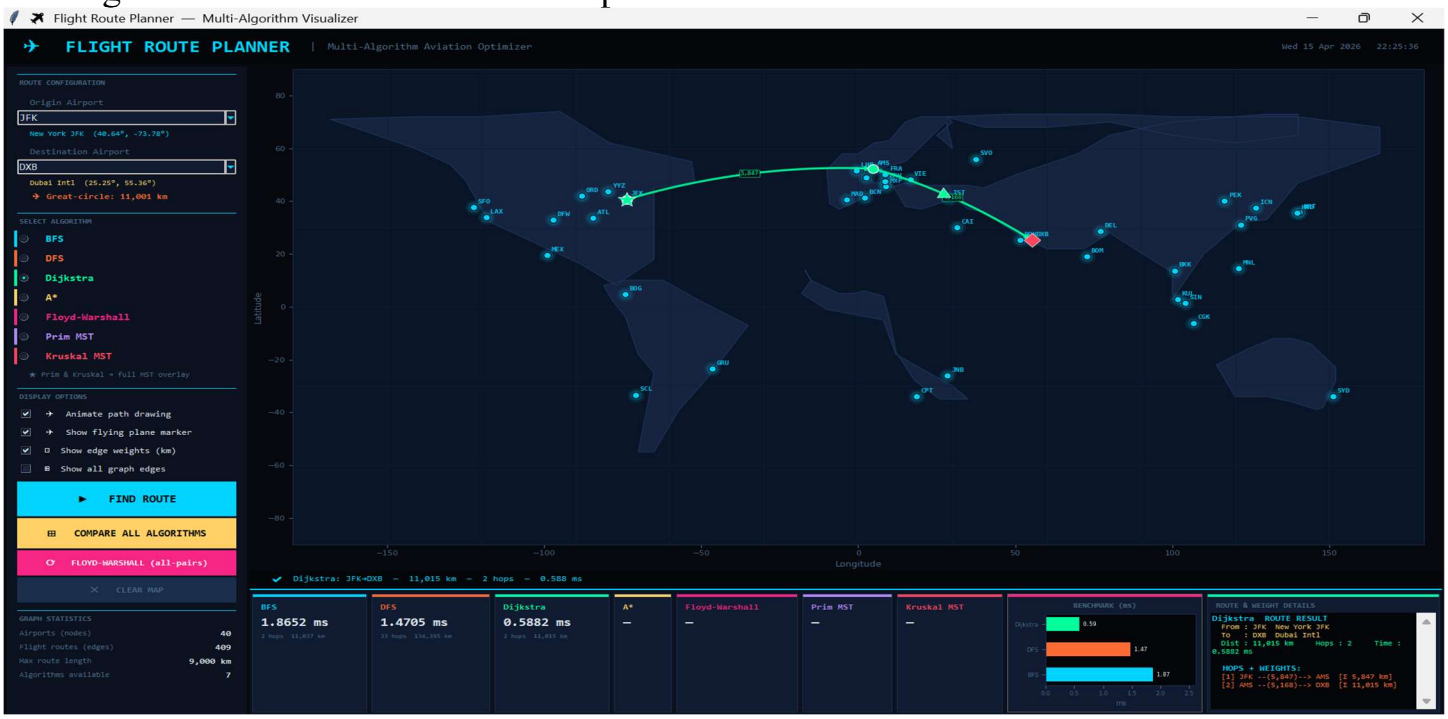
2. DFS (Depth-First Search)

Explores as deep as possible along each branch before backtracking. May not find the shortest path but explores all possibilities systematically.



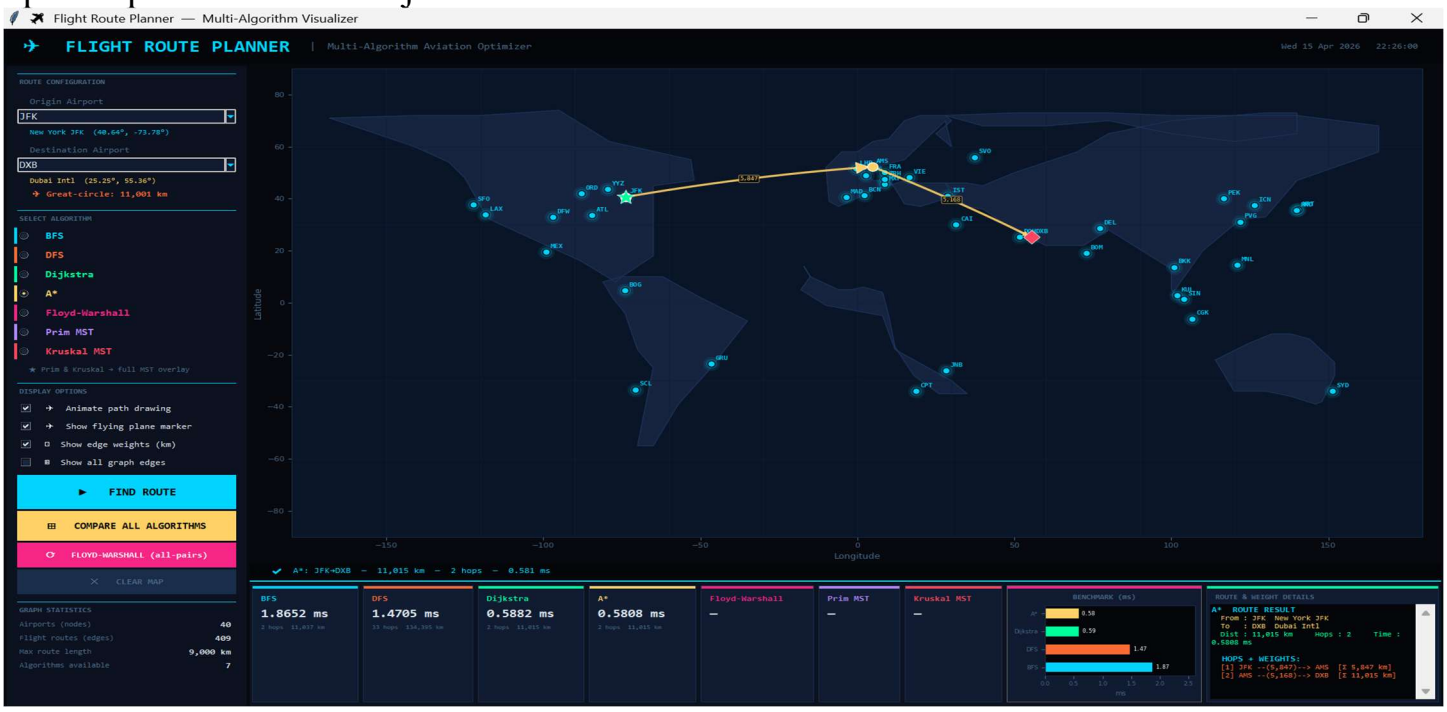
3. Dijkstra's Algorithm

Finds the shortest weighted path using a priority queue. Guarantees the optimal route considering actual great-circle distances between airports.



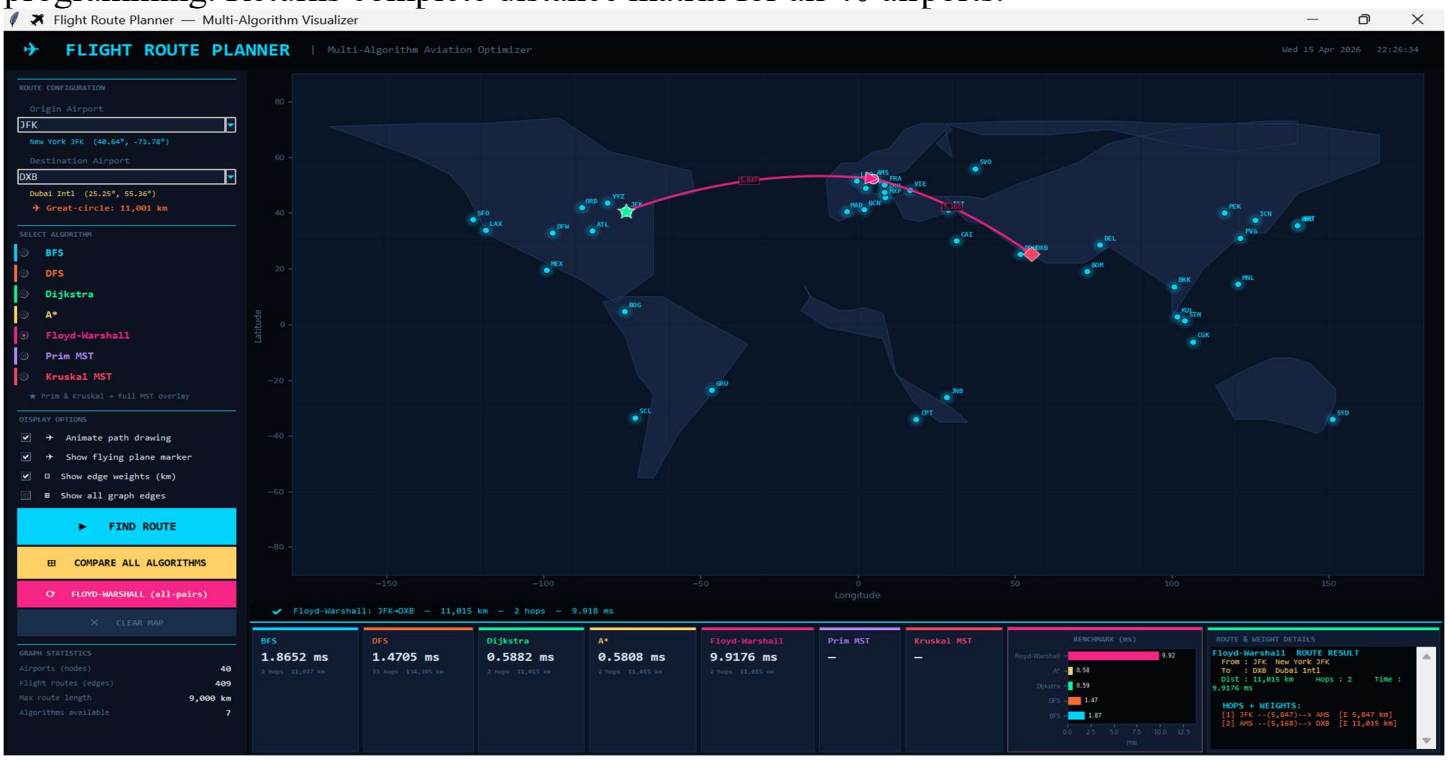
4. A* Search Algorithm

Uses a heuristic function (great-circle distance to destination) to guide the search, finding the optimal path faster than Dijkstra in most cases.



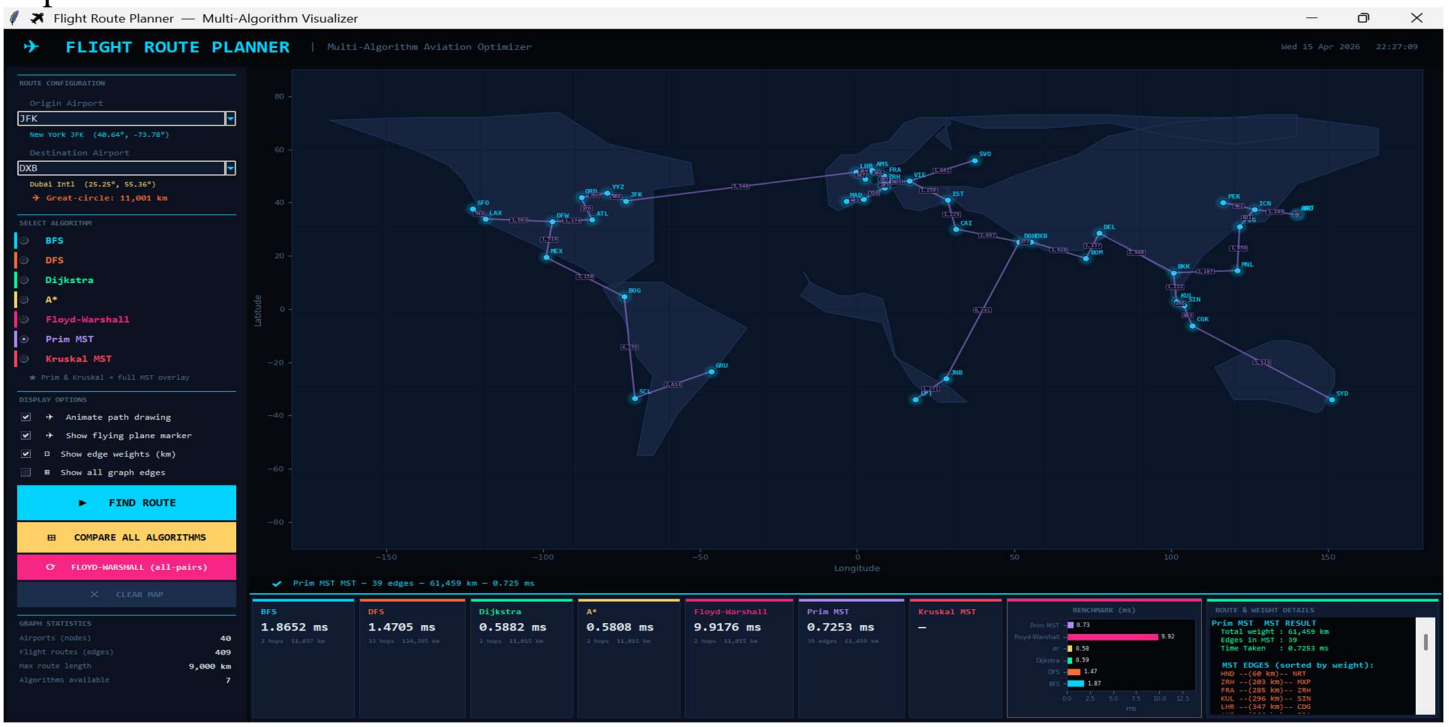
5. Floyd-Warshall Algorithm

Computes shortest paths between all pairs of airports simultaneously using dynamic programming. Returns complete distance matrix for all 40 airports.



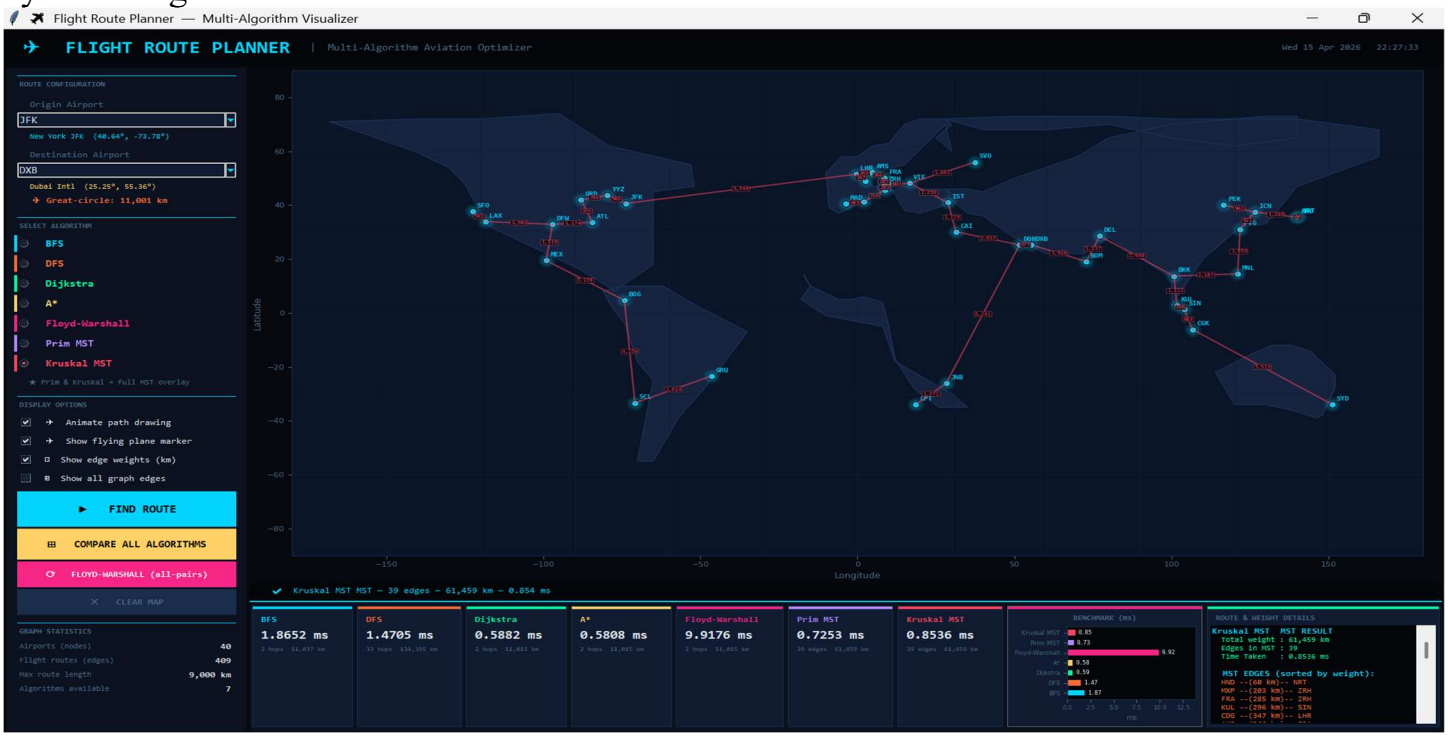
6. Prim's Algorithm (Minimum Spanning Tree)

Builds Minimum Spanning Tree by greedily adding the closest unvisited airport. Connects all 40 airports with minimum total distance.



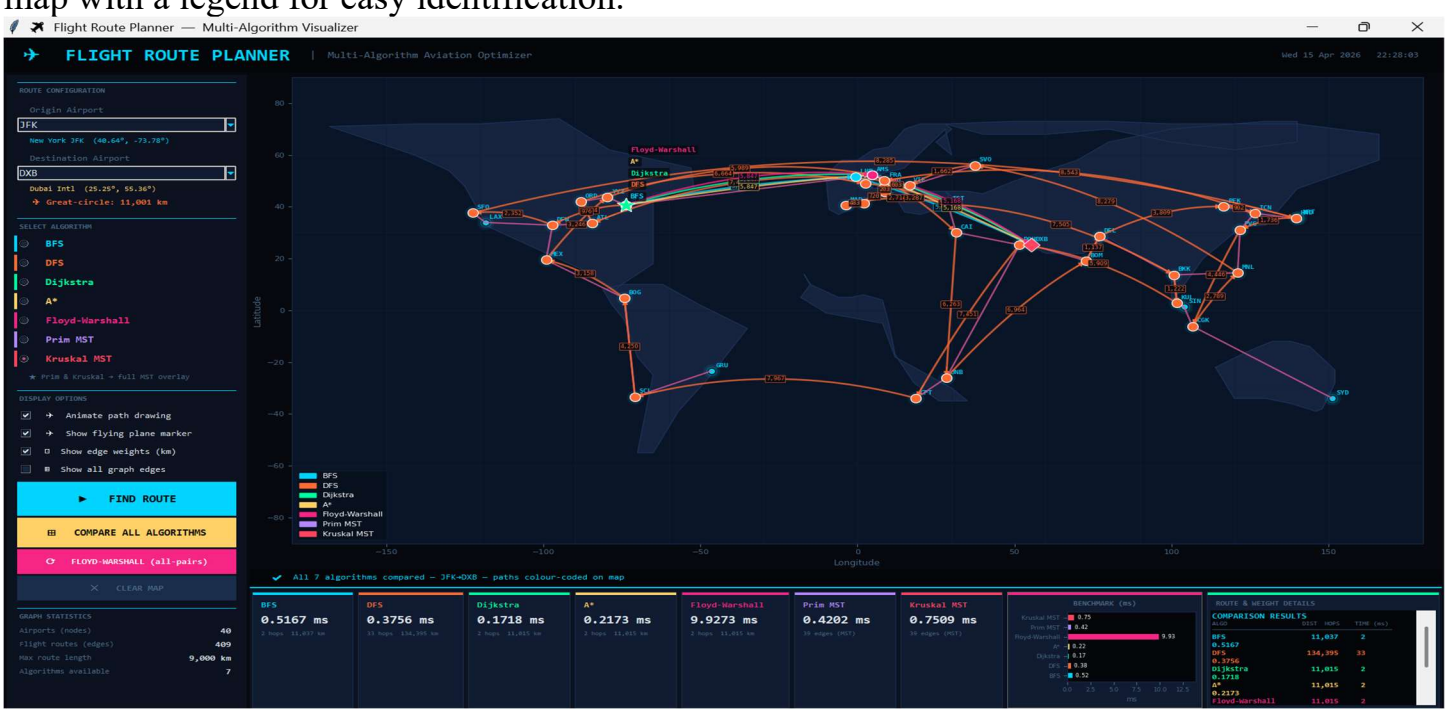
7. Kruskal's Algorithm (Minimum Spanning Tree)

Builds Minimum Spanning Tree by sorting all edges and adding smallest edges that don't create cycles using Union-Find data structure.



Compare All Feature

Runs all 7 algorithms simultaneously and displays each route with a unique color on the same map with a legend for easy identification.



Conclusion

The **Flight Route Planner** project successfully demonstrates the practical implementation of seven graph-based algorithms for aviation route optimization. Built with Python, Tkinter, and Matplotlib, the application provides an interactive platform to visualize and compare BFS, DFS, Dijkstra, A*, Floyd-Warshall, Prim, and Kruskal algorithms using real-world airport data with 40+ airports and 409 flight routes.

The system effectively shows how different algorithmic approaches solve the same routing problem - from simple hop-count minimization (BFS) to weighted optimal paths (Dijkstra, A*), all-pairs shortest paths (Floyd-Warshall), and network connectivity (Prim, Kruskal). Real-time execution time measurements allow users to understand performance trade-offs between algorithms. The "Compare All" feature displays all routes simultaneously with color-coded paths, providing clear visual comparison. This project serves as both an educational tool for understanding graph algorithms and a practical demonstration of their real-world applications in route planning.

Future Work

The Flight Route Planner can be extended with the following features:

1. **Live Weather Integration** - Add real-time weather API to adjust routes based on wind and storms
2. **Multi-Objective Optimization** - Balance fuel, time, cost, and safety simultaneously
3. **3D Globe View** - Upgrade from 2D map to interactive 3D globe
4. **Real Flight Data** - Integrate actual airline schedules and aircraft types
5. **Export Functionality** - Support route export to GPX, KML, and JSON formats
6. **Mobile App Version** - Develop for smartphones and tablets
7. **Blockchain Flight Logging** - Create immutable flight history records

References

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
   - Chapters 22-25 covering BFS, DFS, Dijkstra, Floyd-Warshall, Prim, and Kruskal algorithms
2. **Russell, S., & Norvig, P.** (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
   - Chapter 3: Problem Solving by Searching (A* algorithm and heuristic functions)
3. **Matplotlib Development Team**. (2024). *Matplotlib 3.5 Documentation - Visualization with Python*. https://matplotlib.org/stable/contents.html
4. **OpenFlights**. (2024). *Airport and Airline Route Database*. https://openflights.org/data.html (Airport coordinates and route data reference)
5. **Python Software Foundation**. (2024). *Python 3.8+ Standard Library Documentation*. https://docs.python.org/3/ (Tkinter, heapq, collections modules)

GitHub Link

https://github.com/Nancy5848/Flight-Route-Planner-Multi-Algorithm-Flight-Path-Optimization-System-